Practice Lab Test — MERN Item Manager

Task: You are given a starter MERN Item Manager with `name` and `description`. Your job is to **add a `price` field**, test locally, push to GitHub, and deploy frontend + backend separately.

📁 Project Structure

```
mern-item-manager/
├── backend/
│   ├── models/Item.js
│   ├── routes/items.js
│   ├── server.js
│   ├── .env
│   └── package.json
└── frontend/
    ├── src/
    │   ├── App.jsx
    │   ├── components/ItemForm.jsx
    │   ├── components/ItemList.jsx
    │   └── api.js
    ├── .env
    └── package.json
```

🔧 BACKEND CODE

`backend/package.json`

```json
{
  "name": "item-manager-backend",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
```

```
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.0.3",
    "express": "^4.18.2",
    "mongoose": "^7.0.0"
  },
  "devDependencies": {
    "nodemon": "^2.0.22"
  }
}
```

`backend/.env`

```
MONGO_URI=your_mongodb_atlas_connection_string_here
PORT=5000
```

`backend/server.js` — ✅ GIVEN (starter code)

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
require('dotenv').config();

const app = express();
app.use(cors());
app.use(express.json());

mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('MongoDB connected'))
  .catch(err => console.log(err));

const itemRoutes = require('./routes/items');
```

```
app.use('/api/items', itemRoutes);

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port
${PORT}`));
```

`backend/models/Item.js` — ⚠️ YOU MUST ADD `price`

Starter (given):

```
const mongoose = require('mongoose');

const ItemSchema = new mongoose.Schema({
  name: { type: String, required: true },
  description: { type: String, required: true },
  // TODO: Add price field here
}, { timestamps: true });

module.exports = mongoose.model('Item', ItemSchema);
```

✅ **Completed solution:**

```
const mongoose = require('mongoose');

const ItemSchema = new mongoose.Schema({
  name: { type: String, required: true },
  description: { type: String, required: true },
  price: { type: Number, required: true },   // ← NEW FIELD Y
OU ADD
}, { timestamps: true });

module.exports = mongoose.model('Item', ItemSchema);
```

`backend/routes/items.js` — ✅ GIVEN (starter code, works as-is)

```
const express = require('express');
const router = express.Router();
const Item = require('../models/Item');

// GET all items
router.get('/', async (req, res) => {
  try {
    const items = await Item.find();
    res.json(items);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

// POST create item
router.post('/', async (req, res) => {
  const item = new Item({
    name: req.body.name,
    description: req.body.description,
    price: req.body.price,   // will work once you add price
to the model
  });
  try {
    const newItem = await item.save();
    res.status(201).json(newItem);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

// DELETE item
router.delete('/:id', async (req, res) => {
  try {
    await Item.findByIdAndDelete(req.params.id);
    res.json({ message: 'Item deleted' });
```

```
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

// PUT update item
router.put('/:id', async (req, res) => {
  try {
    const updated = await Item.findByIdAndUpdate(req.params.i
d, req.body, { new: true });
    res.json(updated);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

module.exports = router;
```

🎨 FRONTEND CODE

`frontend/package.json`

```
{
  "name": "item-manager-frontend",
  "version": "0.0.0",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "axios": "^1.4.0",
    "react": "^18.2.0",
    "react-dom": "^18.2.0"
  },
```

```
  "devDependencies": {
    "@vitejs/plugin-react": "^4.0.0",
    "vite": "^4.3.9"
  }
}
```

`frontend/.env`

```
VITE_API_URL=http://localhost:5000/api
```

> ⚠️ When you deploy, update this to your deployed backend URL (e.g. Render URL)

`frontend/src/api.js` — ✅ GIVEN

```
import axios from 'axios';

const API = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
});

export const getItems = () => API.get('/items');
export const createItem = (data) => API.post('/items', data);
export const deleteItem = (id) => API.delete(`/items/${id}`);
export const updateItem = (id, data) => API.put(`/items/${id}
`, data);
```

`frontend/src/components/ItemForm.jsx` — ⚠️ YOU MUST ADD price input

Starter (given):

```
import { useState } from 'react';
import { createItem } from '../api';

export default function ItemForm({ onItemAdded }) {
  const [name, setName] = useState('');
  const [description, setDescription] = useState('');
  // TODO: Add price state here

  const handleSubmit = async (e) => {
    e.preventDefault();
    await createItem({ name, description }); // TODO: add pri
ce to this object
    setName('');
    setDescription('');
    onItemAdded();
  };

  return (
    <form onSubmit={handleSubmit} style={{ marginBottom: '2re
m' }}>
      <h2>Add New Item</h2>
      <div>
        <input
          placeholder="Item name"
          value={name}
          onChange={e => setName(e.target.value)}
          required
        />
      </div>
      <div>
        <input
          placeholder="Description"
          value={description}
          onChange={e => setDescription(e.target.value)}
          required
```

```
        />
      </div>
      {/* TODO: Add price input here */}
      <button type="submit">Add Item</button>
    </form>
  );
}
```

✅ **Completed solution:**

```
import { useState } from 'react';
import { createItem } from '../api';

export default function ItemForm({ onItemAdded }) {
  const [name, setName] = useState('');
  const [description, setDescription] = useState('');
  const [price, setPrice] = useState('');   // ← NEW

  const handleSubmit = async (e) => {
    e.preventDefault();
    await createItem({ name, description, price: Number(pric
e) });  // ← UPDATED
    setName('');
    setDescription('');
    setPrice('');  // ← NEW
    onItemAdded();
  };

  return (
    <form onSubmit={handleSubmit} style={{ marginBottom: '2re
m' }}>
      <h2>Add New Item</h2>
      <div>
        <input
          placeholder="Item name"
          value={name}
```

```
            onChange={e => setName(e.target.value)}
            required
          />
        </div>
        <div>
          <input
            placeholder="Description"
            value={description}
            onChange={e => setDescription(e.target.value)}
            required
          />
        </div>
        <div>
          <input
            placeholder="Price (e.g. 29.99)"
            type="number"
            value={price}
            onChange={e => setPrice(e.target.value)}
            required
          />
        </div>   {/* ← NEW */}
        <button type="submit">Add Item</button>
      </form>
    );
  }
```

`frontend/src/components/ItemList.jsx` — ⚠️ YOU MUST DISPLAY price

Starter (given):

```
import { deleteItem } from '../api';

export default function ItemList({ items, onRefresh }) {
  const handleDelete = async (id) => {
```

```
    await deleteItem(id);
    onRefresh();
  };

  return (
    <div>
      <h2>Items</h2>
      {items.map(item => (
        <div key={item._id} style={{ border: '1px solid #cc
c', padding: '1rem', marginBottom: '1rem' }}>
          <h3>{item.name}</h3>
          <p>{item.description}</p>
          {/* TODO: Display price here */}
          <button onClick={() => handleDelete(item._id)}>Dele
te</button>
        </div>
      ))}
    </div>
  );
}
```

✅ **Completed solution:**

```
import { deleteItem } from '../api';

export default function ItemList({ items, onRefresh }) {
  const handleDelete = async (id) => {
    await deleteItem(id);
    onRefresh();
  };

  return (
    <div>
      <h2>Items</h2>
      {items.map(item => (
        <div key={item._id} style={{ border: '1px solid #cc
```

```
c', padding: '1rem', marginBottom: '1rem' }}>
          <h3>{item.name}</h3>
          <p>{item.description}</p>
          <p><strong>Price: ${item.price}</strong></p>   {/*
← NEW */}
          <button onClick={() => handleDelete(item._id)}>Dele
te</button>
        </div>
      ))}
    </div>
  );
}
```

`frontend/src/App.jsx` — ✅ GIVEN (no changes needed)

```
import { useEffect, useState } from 'react';
import { getItems } from './api';
import ItemForm from './components/ItemForm';
import ItemList from './components/ItemList';

function App() {
  const [items, setItems] = useState([]);

  const fetchItems = async () => {
    const res = await getItems();
    setItems(res.data);
  };

  useEffect(() => {
    fetchItems();
  }, []);

  return (
    <div style={{ maxWidth: '700px', margin: '2rem auto', fon
tFamily: 'sans-serif' }}>
```

```
      <h1>Item Manager</h1>
      <ItemForm onItemAdded={fetchItems} />
      <ItemList items={items} onRefresh={fetchItems} />
    </div>
  );
}

export default App;
```

🚀 Steps to Run Locally

```
# Backend
cd backend
npm install
# add your MongoDB URI to .env
npm run dev

# Frontend (new terminal)
cd frontend
npm install
npm run dev
```

🌐 Deployment Checklist

Step	Tool	What to do
Push code	GitHub	Create public repo, push both frontend & backend folders
Deploy backend	Render.com	New Web Service → connect repo → root dir: `backend` → start cmd: `node server.js` → add `MONGO_URI` env var
Deploy frontend	Netlify	New site → connect repo → base dir: `frontend` → build cmd: `npm run build` → publish dir: `dist` → add `VITE_API_URL` env var with your Render URL

Step	Tool	What to do
Update API URL	Frontend `.env`	Change `VITE_API_URL` to your live Render backend URL before deploying